

CAPSTONE PROJECT

ParaBank

Playwright Automation Test Suite

Enterprise Quality Engineering with Playwright v1.53+

Target Application	ParaBank — parabank.parasoft.com
Framework	Playwright v1.53+
Language	JavaScript (Node.js 20)
Architecture	Page Object Model (POM)
CI/CD	GitHub Actions — Browser Matrix
Reporting	Allure + GitHub Pages
Total Test Cases	139 across 8 services
Browsers	Chromium · Firefox · WebKit
Pass Rate	98.32% (417 executions)

★ 139 Test Cases | 98.32% Pass Rate across 417 Executions | 3 Browsers: Chromium · Firefox · WebKit

Branch: main

Table of Contents

#	Section	Page
1	Project Overview	3
2	Technology Stack	4
3	Project Architecture & Folder Structure	5
4	Page Object Model Design	6
5	Global Setup & Database Initialisation	7
6	Services Under Test — Summary	8
7	Authentication Service (auth.spec.js)	9
8	Account Overview Service (account.spec.js)	10
9	Fund Transfer Service (transfer.spec.js)	11
10	Transaction History Service (transaction.spec.js)	12
11	Bill Payment Service (bill.spec.js)	13
12	Loan Request Service (loan.spec.js)	14
13	User Profile Service (profile.spec.js)	15
14	Internal REST API Service (api.spec.js)	16
15	Test Data Management	17
16	CI/CD Pipeline — GitHub Actions	18
17	Allure Reporting Configuration	19
18	Allure Report — Execution Results	20
19	Key Technical Challenges & Solutions	21
20	Playwright Configuration Reference	22
21	NPM Scripts Reference	23
22	Evaluation Criteria & Project Goals	24
23	Submission Summary	25

1. Project Overview

This capstone project engineers a production-grade, enterprise-level test automation ecosystem targeting ParaBank — a fully functional online banking demo application built and maintained by Parasoft. The framework is implemented in Playwright v1.53+ with JavaScript (Node.js 20) and follows a strict Page Object Model (POM) architecture to enforce clean separation between test logic and page interaction.

Eight core banking services are automated, each with a dedicated Page Object class and spec file. A minimum of 15 test cases are written per service, extended to 20 for Bill Payment and Loan Request to cover additional input-validation scenarios. The total suite delivers 139 automated test cases executed across three browsers: Chromium, Firefox, and WebKit — producing 417 individual executions per full run.

The solution integrates a GitHub Actions CI/CD pipeline with a browser-matrix strategy, Allure HTML reporting with merged cross-browser results, and a global database initialisation step to ensure consistent test state on every run. The Allure report is automatically deployed to GitHub Pages on every push to main.

Project Goals

- Automate 8 banking services with comprehensive positive, negative, and edge-case coverage.
- Execute all tests reliably across Chromium, Firefox, and WebKit.
- Maintain a strict Page Object Model — no selector logic in spec files.
- Integrate Allure reporting with per-browser artifact upload and a merged HTML report.
- Deploy merged Allure report to GitHub Pages on every push to main.
- Handle ParaBank's AngularJS async rendering with robust waitFor strategies.

Result Summary

Metric	Target	Achieved
Services	8 minimum	8
Test Cases	120 minimum	139
Browsers	3 (Chromium, Firefox, WebKit)	3
Pass Rate	> 95%	98.32%
CI/CD	GitHub Actions	Browser matrix — 3 parallel jobs
Reporting	Allure HTML	Allure + GitHub Pages

2. Technology Stack

Every component in the stack was selected to meet enterprise-grade reliability, cross-browser coverage, and CI/CD integration requirements.

Component	Technology / Version	Purpose
Test Framework	Playwright v1.53+	Browser automation across Chromium, Firefox, WebKit
Language	JavaScript (Node.js 20)	Spec files, Page Objects, configuration
Architecture	Page Object Model (POM)	Two-layer separation — locators in pages, logic in specs
Browsers	Chromium · Firefox · WebKit	Cross-browser coverage via matrix CI
CI/CD	GitHub Actions	Browser matrix — 3 parallel jobs per push
Reporting	allure-playwright + allure-commandline	HTML report merged from all three browsers
Report Hosting	GitHub Pages	Auto-deployed on every push to main
Target Site	parabank.parasoft.com	Parasoft demo banking application (AngularJS)
Test Data	fixtures/testData.js	Centralised, no hardcoded strings in spec files
DB Init	global-setup.js / initializeDB	Seeds john/demo account before every run

Why Playwright

Playwright was chosen over Selenium/WebDriver because it provides a unified API across all three browser engines (Blink, Gecko, WebKit) from a single package, built-in auto-waiting that eliminates most explicit waits, native network interception for API testing within the same framework, and first-class TypeScript/JavaScript support with strong tooling integration.

Why Page Object Model

The POM pattern enforces a strict boundary: all CSS selectors and locator logic live exclusively in the pages/ directory. Spec files contain only test logic — describe blocks, expect assertions, and POM method calls. This means when ParaBank updates its DOM structure, only the relevant Page Object class needs updating, not the test cases.

3. Project Architecture & Folder Structure

The project lives inside a `parabank-project/` directory within the `Capstone_Project` repository. The layout enforces a clean monorepo structure with a strict two-layer POM separation.

Path	Type	Description
<code>playwright.config.js</code>	Config	3 browser projects, Allure reporter, timeouts, retries
<code>package.json</code>	Config	Dependencies: <code>@playwright/test</code> , <code>allure-playwright</code>
<code>fixtures/testData.js</code>	Data	All test data — credentials, payee, loan, profile
<code>pages/BasePage.js</code>	Page Object	Base class: <code>this.page + navigate()</code>
<code>pages/AuthPage.js</code>	Page Object	Login, logout, register — locators and actions
<code>pages/AccountPage.js</code>	Page Object	Account overview and account detail
<code>pages/TransferPage.js</code>	Page Object	Fund transfer — smart same-account avoidance
<code>pages/TransactionPage.js</code>	Page Object	Transaction history, filters, amount search
<code>pages/BillPage.js</code>	Page Object	Bill payment form and submission
<code>pages/LoanPage.js</code>	Page Object	Loan request form and submission
<code>pages/ProfilePage.js</code>	Page Object	User profile — pre-filled data and update
<code>tests/global-setup.js</code>	Setup	Calls <code>/initializeDB</code> once before all tests
<code>tests/auth.spec.js</code>	Spec	17 authentication test cases
<code>tests/account.spec.js</code>	Spec	16 account overview test cases
<code>tests/transfer.spec.js</code>	Spec	16 fund transfer test cases
<code>tests/transaction.spec.js</code>	Spec	17 transaction history test cases
<code>tests/bill.spec.js</code>	Spec	20 bill payment test cases
<code>tests/loan.spec.js</code>	Spec	20 loan request test cases
<code>tests/profile.spec.js</code>	Spec	17 user profile test cases
<code>tests/api.spec.js</code>	Spec	16 internal REST API test cases
<code>.github/workflows/playwright.yml</code>	CI/CD	Browser matrix — test + report jobs

4. Page Object Model Design

Every Page Object class extends `BasePage`, which provides exactly two things: `this.page` (the Playwright Page instance) and `navigate(path)` which calls `this.page.goto(path)` resolving against the configured `baseUrl`. No selector string ever appears in a spec file.

BasePage.js

```
class BasePage {  
  constructor(page) { this.page = page; }  
  async navigate(path) { await this.page.goto(path); }  
}  
module.exports = BasePage;
```

Subclass Pattern — BillPage example

```
const BasePage = require('./BasePage');  
class BillPage extends BasePage {  
  constructor(page) {  
    super(page);  
    this.payeeNameInput = page.locator('input[name="payee.name"]');  
    this.amountInput = page.locator('input[name="amount"]');  
    this.sendButton = page.locator('input[value="Send Payment"]');  
    this.fromAccount = page.locator('select[name="fromAccountId"]');  
  }  
  async gotoBillPay() {  
    await this.navigate('/parabank/billpay.htm');  
    await this.page.waitForLoadState('domcontentloaded');  
    await this.payeeNameInput.waitFor({ state: 'visible', timeout: 35000 });  
  }  
}
```

Design Rules

- All CSS/XPath selectors are defined in the constructor of the Page Object subclass.
- Spec files only call Page Object methods — never access locators directly.
- Every goto method ends with an element-level `waitFor` to handle AngularJS async rendering.
- `BasePage` provides only the two shared primitives — it has no test-specific logic.
- Page Object methods return values or perform actions; they never contain `expect()` calls.

5. Global Setup & Database Initialisation

ParaBank's public demo server at parabank.parasoft.com is shared by all users worldwide. Parasoft periodically wipes the database, which removes the john/demo account and all sample data that every test depends on. Without a setup step, tests would fail with login errors or empty account tables on a fresh server state.

Solution: global-setup.js

The global-setup.js file is registered in playwright.config.js as the globalSetup hook. It runs exactly once before any browser launches or any test file is loaded. It sends a GET request to the /initializeDB REST endpoint, which seeds the john/demo account, sample checking and savings accounts, and enough transaction history for all test cases to pass.

```
// tests/global-setup.js
const { request } = require('@playwright/test');
async function globalSetup() {
  const ctx = await request.newContext({ baseURL: 'https://parabank.parasoft.com' });
  try {
    await ctx.get('/parabank/services/bank/initializeDB');
    console.log('Database initialised.');
```

Non-Fatal Design

The setup is intentionally non-fatal. If the endpoint is unreachable, a warning is logged and the test run continues. A prior run may have already seeded the database and it may still be intact.

Registration in playwright.config.js

```
module.exports = defineConfig({
  globalSetup: require.resolve('../tests/global-setup.js'),
  ...
});
```

6. Services Under Test — Summary

Eight core banking services are automated. Each maps to one dedicated Page Object class and one spec file.

#	Service	Page Object	Spec File	Tests	Blocks
1	Authentication	AuthPage.js	auth.spec.js	17	5
2	Account Overview	AccountPage.js	account.spec.js	16	5
3	Fund Transfer	TransferPage.js	transfer.spec.js	16	5
4	Transaction History	TransactionPage.js	transaction.spec.js	17	5
5	Bill Payment	BillPage.js	bill.spec.js	20	6
6	Loan Request	LoanPage.js	loan.spec.js	20	6
7	User Profile	ProfilePage.js	profile.spec.js	17	5
8	Internal REST API	N/A (APIRequestContext)	api.spec.js	16	4
			Total	139	

Test Distribution by Browser

Because all 139 tests run on all three browsers via the CI matrix, each full pipeline run produces 417 test executions.

Browser	Test Executions	Failures (Latest Run)	Pass Rate
Firefox	139	3	97.84%
Chromium	139	2	98.56%
WebKit	139	2	98.56%
Combined	417	7	98.32%

Service Categories

- UI Services (1–7): All use Playwright browser automation via Page Object classes.
- API Service (8): Uses Playwright's APIRequestContext — no browser launched, pure HTTP calls.
- Bill Payment and Loan Request: Extended to 20 tests each with a Block 6 covering pure DOM-attribute and input-validation checks.

7. Authentication Service

Spec File: `auth.spec.js` | **Page Object:** `AuthPage.js` | **Test Cases:** 17

Covers the complete authentication lifecycle: UI verification on the homepage, valid login flow, invalid credential handling (wrong password, wrong username, empty fields, SQL injection attempt), logout, and registration page verification.

Test ID	Description	Block
TC-AUTH-01	Login panel renders on homepage	Block 1 — Page UI
TC-AUTH-02	Username input is visible	Block 1 — Page UI
TC-AUTH-03	Password input is visible	Block 1 — Page UI
TC-AUTH-04	Log In button is visible	Block 1 — Page UI
TC-AUTH-05	Register link is visible	Block 1 — Page UI
TC-AUTH-06	Valid credentials redirect to account overview	Block 2 — Valid Login
TC-AUTH-07	Logout link appears after successful login	Block 2 — Valid Login
TC-AUTH-08	Account Services heading visible after login	Block 2 — Valid Login
TC-AUTH-09	Wrong password shows error message	Block 3 — Invalid Login
TC-AUTH-10	Wrong username shows error message	Block 3 — Invalid Login
TC-AUTH-11	Empty credentials show error message	Block 3 — Invalid Login
TC-AUTH-12	SQL injection attempt does not log in	Block 3 — Invalid Login
TC-AUTH-13	Logout returns to login page	Block 4 — Logout
TC-AUTH-14	URL contains index or login after logout	Block 4 — Logout
TC-AUTH-15	Register page loads at /register URL	Block 5 — Registration
TC-AUTH-16	First name field visible on register page	Block 5 — Registration
TC-AUTH-17	Register button visible on register page	Block 5 — Registration

8. Account Overview Service

Spec File: `account.spec.js` | **Page Object:** `AccountPage.js` | **Test Cases:** 16

Validates the account overview page after login: page load, table visibility, account link navigation, account detail content (number, balance, type, available balance), and navigation back to overview.

Test ID	Description	Block
TC-ACCT-01	Account overview page loads after login	Block 1 — Page Load
TC-ACCT-02	Page title is visible	Block 1 — Page Load
TC-ACCT-03	Account table is visible	Block 1 — Page Load
TC-ACCT-04	At least one account row is present	Block 1 — Page Load
TC-ACCT-05	Account links visible in table	Block 2 — Account Links
TC-ACCT-06	Clicking first account navigates to activity page	Block 2 — Account Links
TC-ACCT-07	Account detail page shows account number	Block 2 — Account Links
TC-ACCT-08	Account detail page shows balance	Block 2 — Account Links
TC-ACCT-09	Account type is displayed	Block 3 — Account Details
TC-ACCT-10	Available balance is displayed	Block 3 — Account Details
TC-ACCT-11	Accounts Overview link visible after login	Block 4 — Navigation
TC-ACCT-12	Clicking overview link stays on overview page	Block 4 — Navigation
TC-ACCT-13	Back to overview from account detail works	Block 4 — Navigation
TC-ACCT-14	Account table content is not empty	Block 5 — Table Content
TC-ACCT-15	Account table has header row	Block 5 — Table Content
TC-ACCT-16	Multiple accounts can exist in table	Block 5 — Table Content

9. Fund Transfer Service

Spec File: `transfer.spec.js` | **Page Object:** `TransferPage.js` | **Test Cases:** 16

Covers the fund transfer page UI, dropdown population, valid transfers (standard, small, decimal amounts), invalid transfers (zero, empty amount), and left-nav accessibility. Includes smart same-account avoidance logic.

Test ID	Description	Block
TC-TRF-01	Transfer page loads after login	Block 1 — UI
TC-TRF-02	Amount input field is visible	Block 1 — UI
TC-TRF-03	From account dropdown is visible	Block 1 — UI
TC-TRF-04	Transfer button is visible	Block 1 — UI
TC-TRF-05	From account dropdown has at least one account	Block 2 — Dropdowns
TC-TRF-06	To account dropdown is visible	Block 2 — Dropdowns
TC-TRF-07	To account dropdown has at least one account	Block 2 — Dropdowns
TC-TRF-08	Valid transfer shows success title	Block 3 — Valid Transfer
TC-TRF-09	Success page shows transfer complete message	Block 3 — Valid Transfer
TC-TRF-10	Small amount transfer succeeds	Block 3 — Valid Transfer
TC-TRF-11	Decimal amount transfer is accepted	Block 3 — Valid Transfer
TC-TRF-12	Zero amount shows error or stays on page	Block 4 — Invalid Transfer
TC-TRF-13	Empty amount field shows error or stays on page	Block 4 — Invalid Transfer
TC-TRF-14	Page content visible after invalid submit	Block 4 — Invalid Transfer
TC-TRF-15	Transfer link in left nav visible after login	Block 5 — Navigation
TC-TRF-16	Transfer page accessible from left nav	Block 5 — Navigation

10. Transaction History Service

Spec File: transaction.spec.js | **Page Object:** TransactionPage.js | **Test Cases:** 17

Validates the account activity page: page load, transaction table presence and content, month/type filter dropdowns, Go button behaviour, amount search input, and navigation between overview and activity pages.

Test ID	Description	Block
TC-TXN-01	Account activity page loads after clicking account	Block 1 — Page Load
TC-TXN-02	Right panel content is visible on activity page	Block 1 — Page Load
TC-TXN-03	Page title contains account related text	Block 1 — Page Load
TC-TXN-04	Account overview link visible on activity page	Block 1 — Page Load
TC-TXN-05	Transaction table is visible on activity page	Block 2 — Table
TC-TXN-06	Transaction table has at least one row	Block 2 — Table
TC-TXN-07	Transaction table content is not empty	Block 2 — Table
TC-TXN-08	Transaction table has Date column header	Block 2 — Table
TC-TXN-09	Activity month dropdown is visible	Block 3 — Filters
TC-TXN-10	Transaction type dropdown is visible	Block 3 — Filters
TC-TXN-11	Go button for filter is visible	Block 3 — Filters
TC-TXN-12	Clicking Go button keeps page on activity URL	Block 3 — Filters
TC-TXN-13	Amount search input area visible	Block 4 — Amount Filter
TC-TXN-14	Find Transactions button is visible	Block 4 — Amount Filter
TC-TXN-15	Searching by amount returns page content	Block 4 — Amount Filter
TC-TXN-16	Back to overview from activity page works	Block 5 — Navigation
TC-TXN-17	Activity page accessible from account overview	Block 5 — Navigation

11. Bill Payment Service

Spec File: bill.spec.js | **Page Object:** BillPage.js | **Test Cases:** 20

Extended from 15 to 20 test cases. Block 6 adds five input-validation checks that are pure client-side — no form submission — ensuring reliability across all three browsers.

Test ID	Description	Block
TC-BILL-01	Bill pay page loads after login	Block 1 — Page Load
TC-BILL-02	Right panel is visible on bill pay page	Block 1 — Page Load
TC-BILL-03	Bill pay page title contains relevant text	Block 1 — Page Load
TC-BILL-04	Payee name input is visible	Block 2 — Form Fields
TC-BILL-05	Address input is visible	Block 2 — Form Fields
TC-BILL-06	Amount input is visible	Block 2 — Form Fields
TC-BILL-07	Account number input is visible	Block 2 — Form Fields
TC-BILL-08	Send Payment button is visible	Block 2 — Form Fields
TC-BILL-09	From account dropdown is visible	Block 3 — From Account
TC-BILL-10	From account dropdown loads accounts	Block 3 — From Account
TC-BILL-11	Submitting valid payment shows response	Block 4 — Payment Submit
TC-BILL-12	Payment response contains payment related text	Block 4 — Payment Submit
TC-BILL-13	Submitting empty form shows page response	Block 4 — Payment Submit
TC-BILL-14	Bill pay link visible in left nav after login	Block 5 — Navigation
TC-BILL-15	Bill pay page accessible via left nav link	Block 5 — Navigation
TC-BILL-16	Payee name input accepts and retains text value	Block 6 — Input Validation
TC-BILL-17	Amount input accepts numeric value	Block 6 — Input Validation
TC-BILL-18	From account dropdown has correct name attribute	Block 6 — Input Validation
TC-BILL-19	Account number input can be cleared and re-entered	Block 6 — Input Validation
TC-BILL-20	Send Payment button has correct input type (submit)	Block 6 — Input Validation

12. Loan Request Service

Spec File: `loan.spec.js` | **Page Object:** `LoanPage.js` | **Test Cases:** 20

Extended from 15 to 20 test cases. Block 6 mirrors the Bill Payment approach: pure DOM-attribute and fill/read checks with no form submission.

Test ID	Description	Block
TC-LOAN-01	Loan request page loads after login	Block 1 — Page Load
TC-LOAN-02	Right panel is visible on loan page	Block 1 — Page Load
TC-LOAN-03	Loan page content contains relevant text	Block 1 — Page Load
TC-LOAN-04	Loan amount input is visible	Block 2 — Form Fields
TC-LOAN-05	Down payment input is visible	Block 2 — Form Fields
TC-LOAN-06	From account dropdown is visible	Block 2 — Form Fields
TC-LOAN-07	Apply Now button is visible	Block 2 — Form Fields
TC-LOAN-08	From account dropdown loads options	Block 3 — Account Dropdown
TC-LOAN-09	Loan amount input accepts numeric value	Block 3 — Account Dropdown
TC-LOAN-10	Applying for loan shows response	Block 4 — Loan Application
TC-LOAN-11	Loan response contains loan related text	Block 4 — Loan Application
TC-LOAN-12	Applying with large amount shows response	Block 4 — Loan Application
TC-LOAN-13	Applying with zero amount shows response	Block 4 — Loan Application
TC-LOAN-14	Loan request link visible in left nav	Block 5 — Navigation
TC-LOAN-15	Loan page accessible via left nav link	Block 5 — Navigation
TC-LOAN-16	Down payment input accepts numeric value	Block 6 — Input Validation
TC-LOAN-17	Loan form has all required input fields	Block 6 — Input Validation
TC-LOAN-18	From account dropdown has correct element id	Block 6 — Input Validation
TC-LOAN-19	Loan amount input can be cleared and re-entered	Block 6 — Input Validation
TC-LOAN-20	Apply Now button has correct input type (submit)	Block 6 — Input Validation

13. User Profile Service

Spec File: `profile.spec.js` | **Page Object:** `ProfilePage.js` | **Test Cases:** 17

Validates the user profile update page: page load, all form field visibility, Angular pre-filling of first and last name, profile update submission, and navigation from left nav.

Test ID	Description	Block
TC-PROF-01	Profile page loads after login	Block 1 — Page Load
TC-PROF-02	Right panel is visible on profile page	Block 1 — Page Load
TC-PROF-03	Profile page contains relevant text	Block 1 — Page Load
TC-PROF-04	First name input is visible	Block 2 — Form Fields
TC-PROF-05	Last name input is visible	Block 2 — Form Fields
TC-PROF-06	Address input is visible	Block 2 — Form Fields
TC-PROF-07	City input is visible	Block 2 — Form Fields
TC-PROF-08	Phone input is visible	Block 2 — Form Fields
TC-PROF-09	Update Profile button is visible	Block 2 — Form Fields
TC-PROF-10	First name field is pre-filled by Angular	Block 3 — Pre-filled Data
TC-PROF-11	Last name field is pre-filled by Angular	Block 3 — Pre-filled Data
TC-PROF-12	Updating profile shows response	Block 4 — Update Profile
TC-PROF-13	Profile update response contains success text	Block 4 — Update Profile
TC-PROF-14	Updating with changed data shows response	Block 4 — Update Profile
TC-PROF-15	Profile link visible in left nav after login	Block 5 — Navigation
TC-PROF-16	Profile page accessible via left nav link	Block 5 — Navigation
TC-PROF-17	State and zip fields are present on profile page	Block 5 — Navigation

14. Internal REST API Service

Spec File: `api.spec.js` | **Page Object:** `N/A (APIRequestContext)` | **Test Cases:** 16

Uses Playwright's `APIRequestContext` to test ParaBank's REST endpoints directly — no browser is launched for this spec file. Tests validate HTTP status codes and JSON response structure.

Test ID	Description	Block
TC-API-01	Login endpoint returns 200 for valid credentials	Block 1 — Login API
TC-API-02	Login response contains customer id	Block 1 — Login API
TC-API-03	Login response contains firstName	Block 1 — Login API
TC-API-04	Invalid login returns non-200 status	Block 1 — Login API
TC-API-05	Accounts endpoint returns 200	Block 2 — Customer Accounts
TC-API-06	Accounts response is an array	Block 2 — Customer Accounts
TC-API-07	At least one account is returned	Block 2 — Customer Accounts
TC-API-08	Each account has an id field	Block 2 — Customer Accounts
TC-API-09	Account detail endpoint returns 200	Block 3 — Account Details
TC-API-10	Account detail contains balance field	Block 3 — Account Details
TC-API-11	Account detail contains type field	Block 3 — Account Details
TC-API-12	Invalid account id returns non-200 status	Block 3 — Account Details
TC-API-13	Transactions endpoint returns 200	Block 4 — Transactions
TC-API-14	Transactions response is an array	Block 4 — Transactions
TC-API-15	Each transaction has an id field	Block 4 — Transactions
TC-API-16	Each transaction has an amount field	Block 4 — Transactions

15. Test Data Management

All test data is centralised in fixtures/testData.js. No hardcoded strings or credentials appear in any spec file.

Category	Keys	Used In
validUser	username, password	All spec files — login step
invalidCredentials	wrongPassword, wrongUsername, emptyBoth, sqlInjection	auth.spec.js Block 3
transferData	validAmount, smallAmount, decimalAmount, zeroAmount	transfer.spec.js
payeeData	name, address, city, state, zipCode, phone, account, amount	bill.spec.js
loanData	validAmount, validDownPayment, largeAmount, largeDownPayment, zeroAmount	loan.spec.js
profileData	firstName, lastName, address, city, state, zipCode, phone	profile.spec.js
updatedProfileData	Same keys — alternate values for re-update test	profile.spec.js TC-PROF-14
endpoints	login(), accounts(), account(), transactions() builders	api.spec.js

Why Centralised Test Data

- Single source of truth — updating the john/demo account only requires one file change.
- Spec files remain readable — method calls use named keys, not raw strings.
- Prevents credential sprawl — passwords and usernames never appear inline in test logic.
- Supports reuse — validUser is imported identically across all 7 UI spec files.

16. CI/CD Pipeline — GitHub Actions

The GitHub Actions workflow implements a two-job strategy that achieves full cross-browser coverage with merged Allure reporting in a single pipeline run.

Job 1: test (Browser Matrix)

Runs in a 3-way matrix strategy across chromium, firefox, and webkit. Each matrix job runs in parallel, installs only its own browser, sets `ALLURE_RESULTS_DIR` to `allure-results-{browser}` to avoid directory collisions.

Job 2: report (Merge + Deploy)

Runs after all three browser jobs complete. Downloads all three `allure-results-*` artifacts, merges them, generates a unified HTML report, and deploys it to GitHub Pages on pushes to main/master.

Setting	Value	Reason
fullyParallel	false	ParaBank server cannot handle concurrent sessions reliably
workers	1	Single worker per browser job — avoids account state collision
retries (CI)	1	One automatic retry for transient network errors
timeout	120,000 ms	Long enough for ParaBank's slow AngularJS page loads
actionTimeout	30,000 ms Chrom / 40,000 ms FF+WK	Firefox/WebKit need more time for AngularJS bootstrap
navigationTimeout	90,000 ms	ParaBank network can be slow on the public demo server
headless	true	Required for Ubuntu GitHub Actions runners
trace	on-first-retry	Captures trace for failures without inflating disk usage
screenshot	only-on-failure	Reduces artifact size on passing runs
video	retain-on-failure	Available for debugging intermittent failures

Workflow Triggers

- Push to any branch.
- Pull request targeting main or master.
- Manual dispatch (`workflow_dispatch`).

17. Allure Reporting Configuration

Allure is integrated via allure-playwright, which hooks into Playwright's reporter interface to capture test results, steps, attachments, and metadata in real time.

Reporter Registration in playwright.config.js

```
reporter: [  
  ['list'],  
  ['allure-playwright', {  
    resultsDir: process.env.ALLURE_RESULTS_DIR || 'allure-results',  
  }],  
],
```

The ALLURE_RESULTS_DIR environment variable is set per browser job in the GitHub Actions matrix. This prevents the three parallel jobs from overwriting each other's results.

Report Generation Flow

Step	Action	Output
1	Each test job runs its browser project	allure-results-{browser}/ with raw JSON
2	Each job uploads its results directory	GitHub Actions artifact (14-day retention)
3	Report job downloads all three artifacts	Three allure-results-* directories on runner
4	Results are merged into allure-results/	Single unified results directory
5	allure generate produces HTML report	allure-report/ directory
6	Report uploaded as artifact	Downloadable from GitHub Actions run page
7	Deployed to GitHub Pages (main only)	Live HTML report at GitHub Pages URL

NPM Scripts for Local Reporting

Script	Command
npm run allure:generate	allure generate allure-results --clean -o allure-report
npm run allure:open	allure open allure-report
npm run allure:report	generate + open in sequence

18. Allure Report — Execution Results

The screenshot below shows the Allure Report generated from the pipeline run on May 30, 2026. The report covers 417 test executions across all three browsers with a combined pass rate of 98.32%.

Metric	Value
Run Duration	31 minutes 09 seconds (12:49:53 to 13:21:03)
Total Executions	417 (139 tests x 3 browsers)
Overall Pass Rate	98.32%
Firefox	136 passed / 3 failed
Chromium	137 passed / 2 failed
WebKit	137 passed / 2 failed
Total Failures	7 across all browsers

The 7 failures are consistent with known flakiness on the shared public ParaBank demo server — primarily caused by periodic server resets and occasional AngularJS rendering races on slow connections.

19. Key Technical Challenges & Solutions

AngularJS Async Rendering

Challenge:

ParaBank's bill payment, loan, transfer, and profile pages are powered by AngularJS. The framework injects form fields into the DOM after `domcontentloaded` fires, so naive assertions fail immediately.

Solution:

Every Page Object's `goto` method ends with an explicit `waitFor` on the first form element rather than relying on load-state events.

```
await this.payeeNameInput.waitFor({ state: 'visible', timeout: 35000 });
```

Transfer: Same-Account Rejection

Challenge:

ParaBank silently rejects transfers where `fromAccountId` equals `toAccountId` with no visible error message, causing TC-TRF-08 and TC-TRF-10 to fail.

Solution:

`TransferPage.transfer()` reads all option values from the `toAccountSelect` and programmatically selects the first value that differs from the current `fromAccountSelect` value before clicking Transfer.

Firefox and WebKit Timeouts

Challenge:

Firefox and WebKit fire `domcontentloaded` significantly earlier than Chromium, before any AngularJS bootstrap code has run.

Solution:

`actionTimeout` and `navigationTimeout` are increased to 40,000 ms and 90,000 ms for Firefox and WebKit. All Page Object `goto` methods use element-level `waitFor`.

Flaky TC-TXN-02 — Right Panel Race Condition

Challenge:

After `clickFirstAccount()` and `domcontentloaded`, ParaBank renders `#rightPanel` asynchronously. A direct `toBeVisible()` check raced against the async render.

Solution:

The assertion is wrapped in `expect().toPass()` with a 20-second retry loop, retrying at polling intervals rather than failing immediately.

```
await expect(async () => { await expect(page.locator('#rightPanel')).toBeVisible();
}).toPass({ timeout: 20000 });
```

Shared Demo Server State

Challenge:

ParaBank's public server is shared globally and its database is periodically wiped by Parasoft, removing the john/demo account.

Solution:

global-setup.js calls GET /parabank/services/bank/initializeDB before any test runs. The call is non-fatal — a warning is logged if it fails.

20. Playwright Configuration Reference

The full `playwright.config.js` defines three browser projects in a matrix, registers the Allure reporter, configures timeouts and retry behaviour, and registers the global setup hook.

Browser Projects

Project	Browser	actionTimeout	navigationTimeout	Notes
chromium	Chromium (Blink)	30,000 ms	90,000 ms	Default timeouts
firefox	Firefox (Gecko)	40,000 ms	90,000 ms	Extended — slower AngularJS bootstrap
webkit	WebKit (WebCore)	40,000 ms	90,000 ms	Extended — slower AngularJS bootstrap

Global Settings

Key	Value
testDir	./tests
fullyParallel	false
forbidOnly	true (CI only)
retries	1 (CI) / 0 (local)
workers	1 (CI) / undefined (local)
timeout	120,000 ms
globalSetup	require.resolve('./tests/global-setup.js')
baseURL	https://parabank.parasoft.com
trace	on-first-retry
screenshot	only-on-failure
video	retain-on-failure
headless	true

21. NPM Scripts Reference

All test execution and reporting commands are available as npm scripts.

Test Execution Scripts

Script	Full Command	Purpose
npm test	playwright test	Run all tests across all configured browser projects
npm run test:chromium	playwright test --project=chromium	Run all tests on Chromium only
npm run test:firefox	playwright test --project=firefox	Run all tests on Firefox only
npm run test:webkit	playwright test --project=webkit	Run all tests on WebKit only

Allure Reporting Scripts

Script	Full Command	Purpose
npm run allure:generate	allure generate allure-results --clean -o allure-report	Generate HTML report from raw results
npm run allure:open	allure open allure-report	Open generated HTML report in browser
npm run allure:report	generate + open in sequence	Full generate and open pipeline

Useful Debug Flags

Flag	Purpose
--headed	Run with visible browser window
--debug	Open Playwright Inspector for step-by-step debugging
--trace on	Record trace for every test
--project=chromium	Restrict run to a single browser
--grep TC-AUTH	Run only tests whose title matches the pattern
--reporter=list	Plain text terminal output without Allure

22. Evaluation Criteria & Project Goals

All capstone evaluation criteria have been met or exceeded.

Criterion	Requirement	Delivered	Status
Services	Minimum 8	8 services	Met
Test Cases	Minimum 120	139 total	Met — 139/120
Architecture	Page Object Model	Strict POM — two-layer separation	Met
Browsers	Chromium + Firefox + WebKit	All 3 via matrix CI	Met
CI/CD Pipeline	GitHub Actions workflow	Browser matrix — 3 parallel jobs	Met
HTML Report	Allure reporter	Allure + GitHub Pages deployment	Met
Dual Project	POM + SVG Graphics Engine	Both scaffolded	Met

Exceeded Requirements

- Test case count: 139 delivered vs 120 required — 19 additional tests.
- Bill Payment and Loan Request each extended to 20 tests with Block 6 input-validation coverage.
- Non-fatal global setup with informative warning on initializeDB failure.
- Smart same-account avoidance in TransferPage to prevent silent rejection failures.
- `expect().toPass()` retry pattern for async panel rendering.
- Per-browser timeout tuning (Chromium vs Firefox/WebKit) rather than a single global value.
- Allure results segregated by browser to prevent artifact overwrites in parallel CI jobs.

Test Coverage by Category

Category	Test Cases	Services Covered
Page Load & UI Verification	34	All 8 services
Valid / Happy Path	27	Auth, Transfer, Bill, Loan, Profile, API
Invalid / Negative / Edge Case	28	Auth, Transfer, Bill, Loan, API
Navigation	18	All UI services
DOM Attribute & Input Validation	10	Bill Payment, Loan Request
REST API (HTTP + JSON)	16	api.spec.js
Total	139	8 services

23. Submission Summary

Complete submission reference for the ParaBank Playwright Capstone Project.

Repository Details

Item	Detail
Repository	Capstone_Project
Branch	main
Target Application	https://parabank.parasoft.com
Report URL	GitHub Pages — deployed on every push to main
Workflow File	.github/workflows/playwright.yml

Deliverable Checklist

Deliverable	Location	Status
Playwright config	parabank-project/playwright.config.js	Complete
BasePage	parabank-project/pages/BasePage.js	Complete
8 Page Object classes	parabank-project/pages/	Complete
8 spec files (139 tests)	parabank-project/tests/	Complete
Global setup	parabank-project/tests/global-setup.js	Complete
Test data fixtures	parabank-project/fixtures/testData.js	Complete
GitHub Actions workflow	.github/workflows/playwright.yml	Complete
Allure report (merged)	Deployed to GitHub Pages	Complete
Capstone documentation	Doc/ folder in repository	Complete

Final Statistics

Metric	Value
Total test cases written	139
Total browser executions per run	417 (139 x 3 browsers)
Pass rate — May 30, 2026 run	98.32%
Run duration	31 minutes 09 seconds

Page Object classes	9 (BasePage + 8 service classes)
Spec files	8
Browsers covered	3 — Chromium, Firefox, WebKit
CI jobs per push	4 (3 test + 1 report)
Allure deployment	GitHub Pages — automatic on push to main